

HLS Optimisation & Float / Fixed Intro

L6 · Week 3 · Hardware / Software Codesign 2025/26

José T. de Sousa · IST

HLS IP

Pragmas

Float
Hardware

ap_fixed

Making your IP faster — and understanding what data types cost in silicon

Where We Are

L3 · AXI-Lite

register r/w · small payloads

$\leq 4 \text{ bytes} / \text{transfer}$

L4 · AXI-Stream

streaming · pipelined

any size, but needs CPU write loop

L5 · DMA

bulk transfer · CPU-free

MByte arrays in one command

L6 — Today · Inside the IP: make it faster, choose the right data type

The interface chain is complete. Today we zoom inside the IP and ask: how fast can it go, and what does float actually cost?

Today's Two Questions

(1) Can we make the IP faster?

Run faster without changing what it computes?

- ▶ Initiation Interval (II) — limits throughput
- ▶ PIPELINE, UNROLL, ARRAY_PARTITION
- ▶ Reading the HLS synthesis report

Goal: $II = 1$ on the inner loop.

(2) What does float cost in HW?

A float adder is a multi-stage pipeline:
~400 LUTs, not a single gate.

- ▶ Why the FP accumulator gets $II = 4$
- ▶ `ap_fixed<W,I>` as the fix
- ▶ Area vs. accuracy trade-off (L7 preview)

Goal: know the cost before choosing a type.

Latency vs. Throughput: What Is II?

Latency = 3 cy



II = 2 cy



Latency

Cycles to complete *one* iteration.

Initiation Interval (II)

Cycles between *starting* consecutive iterations:

$$\text{Throughput} = \frac{1}{\text{II} \times T_{\text{clk}}}$$

Goal: II = 1.

What the HLS Scheduler Does

C source (inner loop):

```
for(int i=0; i<N; i++){
    acc += a[i] * b[i];
    // ^load ^load ^mul
    //
    ^accumulate (FP add)
}
```

HLS maps each op to a clock cycle, respecting:

- ▶ Operator latencies (FP add: 4 cycles)
- ▶ Data dependencies

Resulting schedule (float, II=4):

	cy 1	cy 2	cy 3	cy 4	cy 5	cy 6	cy 7
<i>load a[i]</i>	rd						
<i>load b[i]</i>	rd						
<i>multiply</i>		mul	...				
<i>FP add</i>				S1	S2	S3	S4

The FP adder runs cy 4→7 (**4 stages**). New acc ready at cy 7 — next iter's FP add can't start until cy 8. **II = 4.**

What Limits II: Two Main Culprits

(1) Loop-Carried Dependency

An operation depends on a result from the *previous* iteration:

```
// acc carries iter-to-iter
acc += a[i] * b[i];
```

The FP adder takes **4 cycles** to produce the new acc. The next iteration must wait $\Rightarrow$ **II = 4**.

Fix: ap_fixed (II=1) or partial sums + tree reduction.

(2) Memory Port Conflict

A BRAM has **2 ports**. Reading more than 2 arrays per cycle stalls the pipeline:

```
// 3 reads/cycle -> stall
c[i] = a[i]+b[i]+d[i];
```

HLS serialises the extra accesses $\Rightarrow$ II increases.

Fix: ARRAY_PARTITION to give each array its own ports.

Reading the HLS Synthesis Report

Vitis HLS console / Solution Summary:

```
+ Loop Information
+ loop_i
* Trip count : 64
* Latency (clock cycles)
- min/max : 259 / 259
* Initiation interval
- achieved : 4
- target : 1
* Pipeline depth : 7

+ Utilization Estimates
DSP48E : 3
BRAM_18K: 2
LUT/FF : 1842 / 976
```

achieved II = 4, target = 1

HLS could not meet the requested II.
Look for the reason in the dependency report below.

Latency = $(N-1) \times II + d_{\text{body}}$
= $63 \times 4 + 7 = \mathbf{259}$ cycles ✓
 $d=7$: ld(1) + mul(2) + FP-add(4); flush
= $d - II = 3$ cycles.

Also check:

- ▶ **DSP** — multipliers inferred?
- ▶ **LUT/FF** — FP operator cost
- ▶ **BRAM** — arrays on BRAM?

Demo: Baseline Matrix-Multiply Inner Loop

The code HLS sees:

```
void gemm(float A[][N],
          float B[][N],
          float C[][N]){
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++){
            float acc=0;
            for(int k=0;k<N;k++)
                acc+=A[i][k]*B[k][j];
            C[i][j]=acc;
        }
}
```

What HLS reports (no pragmas):

Loop	II	Reason
inner k	4	FP add latency
middle j	259+1	sequential
outer i	—	sequential

Total cycles $\approx N^3 \times 4$

For $N = 64$: $64^3 \times 4 = 1,048,576$ cycles $\approx$ 10 ms @ 100 MHz.

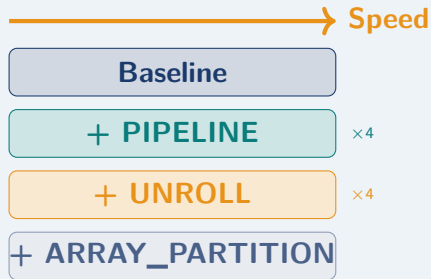
Adding PIPELINE to the inner loop is the first step — next slides show how.

Section 3 — Loop Optimisation Pragmas

Three pragmas, most of the gains:

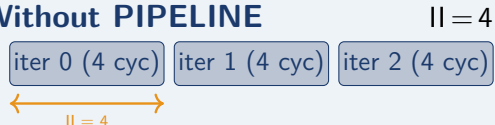
- `#pragma HLS PIPELINE`
- `#pragma HLS UNROLL`
- `#pragma HLS ARRAY_PARTITION`

Standard C/C++ — pragmas are ignored by the compiler, acted on by HLS.

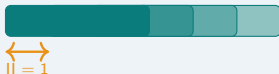


HLS PIPELINE — Concept

Without PIPELINE



With PIPELINE II = 1 (no loop-carried dep.)



II = 1 only when output does not feed back into the next iteration.

Float accumulation (`acc += ...`) gives II = 4 (FP adder latency).

```
#pragma HLS PIPELINE II=1
```

Place inside the loop body.

Param	Meaning
II=N	Target II (default 1)
rewind	Overlap last/first iteration
enable_flush	Drain pipeline on stall

Rule of thumb: start with the innermost loop.

PIPELINE Applied to GEMM

```
for(int i=0;i<N;i++)
  for(int j=0;j<N;j++){
    float acc=0;
    for(int k=0;k<N;k++){
      #pragma HLS PIPELINE
      acc+=A[i][k]*B[k][j];
    }
    C[i][j]=acc;
  }
```

Synthesis report:

Loop	II	Latency
inner k	4	259 cyc
middle j	—	$64^2 \times 259$
outer i	—	$64^3 \times 259$

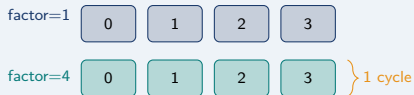
$\approx 1.7\times$ speedup (FP dep. limits II)

1,835,008 $\rightarrow$ 1,060,864 cycles

18.4 ms $\rightarrow$ 10.6 ms @ 100 MHz

HLS UNROLL — Loop Unrolling

`#pragma HLS UNROLL factor=F`
 F iterations execute in parallel each cycle.



Cost: $\text{LUT/DSP} \times F$.

```
for(int k=0;k<N;k++){  
    #pragma HLS PIPELINE II=1  
    #pragma HLS UNROLL factor=4  
    acc+=A[i][k]*B[k][j];  
}
```

Config	II	k-lat.
Baseline	7	448
+ PIPELINE	4	259
+ UNROLL×4	4	70

HLS ARRAY_PARTITION — Memory Bottleneck

Problem: BRAMs have **2 ports max**.
Unrolled loop needs F reads/cycle —
HLS raises II.

```
#pragma HLS ARRAY_PARTITION
```

```
variable=A dim=2 factor=4 cyclic
```

cyclic: interleaved **block:** contiguous

complete: full registers

```
#pragma HLS ARRAY_RESHAPE
```

Wider word, fewer BRAMs — same bandwidth
as partition.

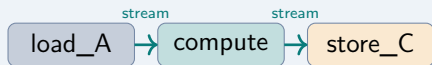
```
#pragma HLS ARRAY_PARTITION  
    variable=A factor=4 cyclic dim=2  
#pragma HLS ARRAY_PARTITION  
    variable=B factor=4 cyclic dim=1
```

Config	II	Speed
Baseline	7	1×
+ PIPELINE	4	≈1.7×
+ UNROLL, no PART.	4!	stalled
+ PART. + ap_fixed	1	7×



HLS DATAFLOW — Task-Level Pipelining

Overlaps *sequential functions* —
producer/consumer at the task level.



```
#pragma HLS DATAFLOW
```

Place at the top-level function.

Use `hls::stream<>` between stages.

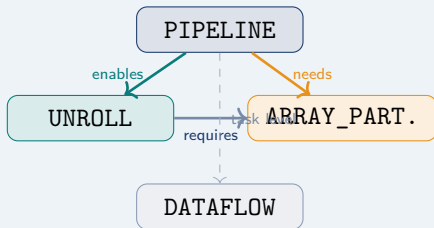
```
void top(data_t in[],
         data_t out[]){
    #pragma HLS DATAFLOW
    hls::stream<data_t> s1,s2;
    load(in,  s1);
    compute(s1, s2);
    store(s2, out);
}
```

Interval = max(stage II), not sum.

Use when: streaming I/O, multi-stage pipelines, DMA + compute overlap.

Pragma Interaction & Guidelines

Situation		Pragma
Loop-carried dep. limits II		PIPELINE
Need throughput	$F \times$	UNROLL factor=F
Too much area		reduce factor
Memory port conflict		ARRAY_PARTITION
Sequential stages		DATAFLOW



**Always pair
UNROLL + PARTITION.**

Unrolling alone serialises memory accesses.

Reading the Vitis HLS Synthesis Report

Field	What to check
Target II	What you requested
Achieved II	What HLS managed
Violation	Dep. or resource conflict
BRAM/DSP/LUT	Resource budget
Slack	Timing (≥ 0)

In Vitis HLS GUI:

Solution → Open Report → Synthesis
Loop tab: per-loop II and latency

Workflow:

1. Synthesise without pragmas
2. Add PIPELINE II=1, check achieved II
3. II > 1? Check Violation:
 - FP dep. → DSP chaining
 - Memory → ARRAY_PARTITION
4. Add UNROLL, re-check area

Best Float Performance

```
void gemm(float A[N][N],
          float B[N][N],
          float C[N][N]){
#pragma HLS ARRAY_PARTITION
    variable=A factor=4 cyclic dim=2
#pragma HLS ARRAY_PARTITION
    variable=B factor=4 cyclic dim=1
    for(int i=0;i<N;i++){
        for(int j=0;j<N;j++){
            float acc=0;
            for(int k=0;k<N;k++){
                #pragma HLS PIPELINE II=1
                #pragma HLS UNROLL factor=4
                acc+=A[i][k]*B[k][j];
            }
            C[i][j]=acc;
        }
    }
}
```

Version	Cycles	Time
Baseline	1,835,008	18.4 ms
+ PIPELINE	1,060,864	10.6 ms
(II=4)		
+ UNROLL×4	≈ 307,200	≈ 3.1 ms
+ PARTITION	≈ 307,200	≈ 3.1 ms

DSP48: ≈36 BRAM: ≈6 LUT: ≈
4×

≈7× speedup (float, II=4), 4×
resources. *ap_fixed* gives another 4×.

Loop Optimisation Pragmas — Key Takeaways

1. PIPELINE — overlap iterations, target $II=1$

2. UNROLL — F iterations in parallel, $F \times$ area

3. ARRAY_PARTITION — split BRAM into F banks

4. DATAFLOW — overlap sequential functions

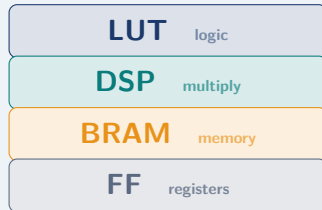
Pragma	II effect
PIPELINE	$II \rightarrow \min \text{ dep. (4 for FP)}$
$UNROLL \times F$	$\downarrow F \times$
ARRAY_PART.	enables above
DATAFLOW	task overlap

Golden rule: measure first, then add pragmas.

Section 4 — Resources & Timing

What we cover:

- FPGA resource types (LUT, DSP, BRAM, FF)
- How HLS maps C operators to resources
- Reading the utilisation report
- Clock constraint & timing closure



Pragmas change speed — but they also change area. Understanding both is essential.

FPGA Resource Types

LUT — Look-Up Table

Implements any Boolean function of ≤ 6 inputs. Used for logic, muxes, adders, comparators.

DSP48E — Multiply-Accumulate block

Hard 18×27 -bit multiplier + adder. HLS infers these automatically for * and MAC.

BRAM_18K — Block RAM

18 Kb dual-port RAM. Arrays > a few elements are mapped here by HLS.

FF — Flip-Flop

1-bit register. Used for pipeline registers, interface signals, state machines.

Rough cost per operator:

Operator	Typical cost
int add/cmp	~1 LUT/bit
int mul	1–3 DSP48E
FP add (32-bit)	~400 LUT, 2 DSP
FP mul (32-bit)	~3 DSP48E

Reading the Utilisation Report

Vitis HLS — Utilization Estimates:

+ Utilization Estimates

+ Summary

Name : gemm

BRAM_18K : 6

DSP48E : 9

FF : 1204

LUT : 3841

+ Details

* A[64][64] -> BRAM x2

* B[64][64] -> BRAM x2

* C[64][64] -> BRAM x2

* fmul_32ns -> DSP48E x3

* fadd_32ns -> DSP48E x6

6 BRAM — one per matrix

Each 64×64 float = 16 KB > 1
BRAM_18K, so HLS uses 2 per array.

9 DSP — FP operators

FP mul = 3 DSP, FP add = 6 DSP.
Float is expensive.

Check against your device budget.

Zynq-7020: 220 DSP, 140 BRAM —
plenty here.

Clock Constraint & Timing Closure

Set the clock in the HLS solution:

Target period = 10 ns $\Rightarrow$ 100 MHz.

HLS adds pipeline registers to meet timing.

What HLS reports:

```
+ Timing (ns)
* Summary
Clock target : 10.00
Estimated : 8.24
Uncertainty : 1.25
```

Estimated < target: timing met.

If estimated > target:

HLS still generates RTL but flags a warning. Vivado implementation may then fail timing — reduce the clock or simplify the datapath.

Typical causes of timing failure:

- ▶ FP operators on critical path
- ▶ Wide multipliers not absorbed by DSP
- ▶ Long combinational mux trees

Resources vs. Speed: the Trade-off

Action	Speed	Area
PIPELINE	$\uparrow\uparrow$	+FF regs
UNROLL $\times F$	$\uparrow F \times$	$\uparrow F \times$
ARRAY_PART.	enables UNR	+BRAM
float $\rightarrow$ int	$\uparrow\uparrow$	$\downarrow\downarrow$
Reduce clock	$\downarrow$	$\searrow$

Rule: PIPELINE first — FF overhead is small. UNROLL only if you have DSP/BRAM headroom.

Typical workflow:

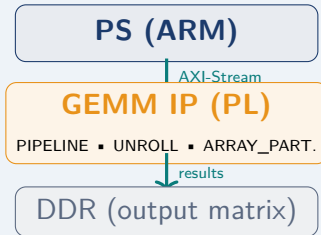
1. Compile baseline, note II & area.
2. Add PIPELINE to inner loop.
3. Check report — II met?
4. Memory bottleneck? Add ARRAY_PART.
5. If still slow: UNROLL, watch area.
6. Verify timing in Vivado.

Section 5 — Connecting to the Lab

Lab goal: apply today's pragmas to a real GEMM HLS IP and measure the effect.

- Baseline inner loop — $II = 7$
- Add PIPELINE — $II = 4$ (FP dep.)
- Add UNROLL + ARRAY_PARTITION
- Read the synthesis report each time

You already have the AXI-Stream interface from L5. Today you optimise what's inside the IP.



Lab Steps

Step 1 — Baseline (no pragmas):

```
for(int k=0; k<N; k++)  
    acc += A[i][k]*B[k][j];
```

Note II, latency, DSP, LUT in report.

Step 2 — Add PIPELINE:

```
for(int k=0; k<N; k++){  
    #pragma HLS PIPELINE II=1  
    acc += A[i][k]*B[k][j];  
}
```

Does II reach 1? Check the report.

Step 3 — Add UNROLL + ARRAY_PARTITION:

```
#pragma HLS ARRAY_PARTITION \  
    variable=A cyclic factor=4  
#pragma HLS ARRAY_PARTITION \  
    variable=B cyclic factor=4  
for(int k=0; k<N; k++){  
    #pragma HLS PIPELINE II=1  
    #pragma HLS UNROLL factor=4  
    acc += A[i][k]*B[k][j];  
}
```

Record II, latency & DSP count after each step.

Expected Results

Version	II	DSP	Sp
Baseline	7	9	1×
+ PIPELINE (float)	4	9	≈ 1
+ ap_fixed + UNROLL×4	1	36	≈ 2

If $II > 1$ after PIPELINE, look for:

- ▶ Loop-carried FP dependency (expected)
- ▶ Memory port conflict (add ARRAY_PART.)
- ▶ Report says “ $II = 4$, target = 1”

What to hand in:

- Synthesis report screenshot per step
- Table: II, latency, DSP per version
- Brief explanation of why II changes

Common mistake:

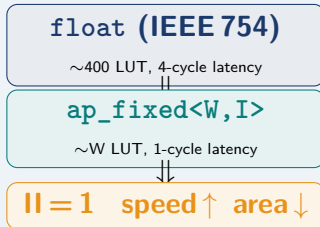
UNROLL without ARRAY_PARTITION
— memory ports stall the pipeline. Always pair them.

Section 6 — Float vs. Fixed

What we cover:

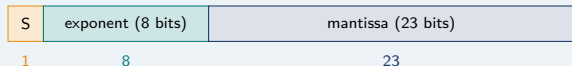
- What IEEE 754 float actually is in HW
- Why float accumulation gives $II = 4$
- `ap_fixed<W,I>` — format & syntax
- Why fixed-point gives $II = 1$
- Area, accuracy, and how to choose W, I

*Float is convenient. Fixed is fast and small.
Knowing the cost lets you choose.*



What IEEE 754 Float Actually Is

A 32-bit float is split into three fields:



$$\text{Value} = (-1)^S \times 1.M \times 2^{E-127}$$

To add two floats in hardware:

1. Align exponents (shift mantissa)
2. Add the mantissas
3. Normalise the result
4. Round

This takes 4 pipeline stages.

Each stage is a clock cycle. The result of `acc += x` is not available until 4 cycles later.

⇒ Loop-carried dependency forces $II = 4$.

*A float multiplier is similarly expensive:
~3 DSP48E, 3-cycle latency.*

Float Hardware Cost

Xilinx 7-series resources for one operator:

Operator	LUT	DSP	Latency
float add	~400	2	4 cy
float mul	~150	3	3 cy
float div	~550	0	28 cy
int32 add	~32	0	1 cy
int32 mul	~0	3	1 cy

Key takeaway: one FP adder costs as much as ~12 full adders in LUTs.

Why does it matter?

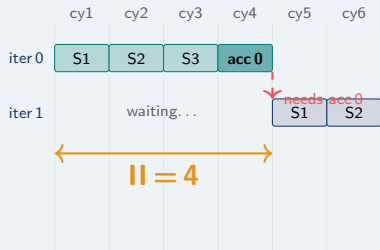
In a GEMM inner loop with UNROLL $\times 4$:

- 4 FP adders = $4 \times 400 = 1600$ LUT
- 4 FP mul = $4 \times 3 = 12$ DSP
- Total: >2000 LUT for one unrolled loop

Compare: `ap_fixed<16,8>` adder ≈ 16 LUT.

Float is fine for small designs. At high unroll factors or tight area budgets, fixed-point is essential.

Why Float Accumulation Forces $II = 4$



The dependency chain:

`acc += x` reads `acc` and writes a new `acc`.

The new value is ready after 4 cycles.

The next iteration needs that value

$\Rightarrow$ must wait 4 cycles $\Rightarrow II = 4$.

Fix: use `ap_fixed`.

A fixed-point adder has **1-cycle latency**.

Dependency resolves in 1 cycle $\Rightarrow II = 1$.

Introducing ap_fixed<W,I>

Syntax:

```
#include "ap_fixed.h"
```

```
// W = total bits, I = integer bits
```

```
ap_fixed<16, 8> x;
```

```
// 8 integer bits, 8 fraction
```

```
// range: -128 to +127.996
```

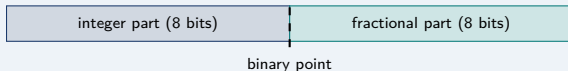
```
// resolution: 1/256
```

Vitis HLS includes ap_fixed.h.

Works in simulation and synthesis.

Drops in as a replacement for float.

Bit layout for ap_fixed<16,8>:



Common rounding modes:

AP_RND — round to nearest

AP_TRN — truncate (default, cheapest)

AP_SAT — saturate on overflow

Fixed-Point Accumulator: II = 1

Change one type:

```
// Before: float  
float acc = 0;  
for(int k=0; k<N; k++){  
    #pragma HLS PIPELINE II=1  
    acc += A[i][k]*B[k][j];  
}
```

```
// After: ap_fixed  
ap_fixed<32,16> acc = 0;  
for(int k=0; k<N; k++){  
    #pragma HLS PIPELINE II=1  
    acc += A[i][k]*B[k][j];  
}
```

Synthesis report:

	float	ap_fixed
II achieved	4	1
LUT	1842	312
DSP	9	6
Latency (cy)	257	66

4× faster, 6× fewer LUTs.

The only change was the type of acc. HLS infers a 1-cycle adder automatically.

Choosing W and I

I — integer bits (range):

Enough to hold the max value without overflow.

$I \geq \lceil \log_2(\text{max}) \rceil + 1$ (sign).

E.g. $\text{max} = 64 \Rightarrow I = \lceil \log_2(64) \rceil + 1 = 7$.

W—I — fractional bits (precision):

More bits = lower quantisation error.

Pick smallest W that meets your accuracy spec.

Start with `ap_fixed<16,8>`.

Area scales with W:

Type	LUT	II
<code>ap_fixed<8,4></code>	~8	1
<code>ap_fixed<16,8></code>	~16	1
<code>ap_fixed<32,16></code>	~32	1
<code>float (32-bit)</code>	~400	4

Workflow: simulate with float first, measure the dynamic range, then pick W and I.

Accuracy & Quantisation

Reducing W introduces **quantisation error**:

$$\text{Quantisation step} = 2^{-(W-I)}$$

For `ap_fixed<16,8>`: $\text{step} = 2^{-8} \approx 0.004$.

Values are rounded to the nearest multiple.

When accuracy matters:

- ▶ Neural network weights: 8-bit often fine
- ▶ DSP filters: 16–18 bits typical
- ▶ Scientific computing: float or double

Verification approach:

1. Run C simulation with float (golden reference).
2. Switch to `ap_fixed` and re-simulate.
3. Compare outputs — measure MSE or max error.
4. Increase W if accuracy insufficient.
5. HLS co-simulation confirms HW matches SW.

Vitis HLS runs the same C++ test bench for both software and RTL simulation.

Code: Replacing float with ap_fixed

Before (float):

```
void gemm(float A[][N],
          float B[][N],
          float C[][N]){
    for(int i=0;i<N;i++){
        for(int j=0;j<N;j++){
            float acc=0.0f;
            for(int k=0;k<N;k++){
                #pragma HLS PIPELINE
                acc+=A[i][k]*B[k][j];
            }
            C[i][j]=acc;
        }
    }
}
```

After (ap_fixed):

```
#include "ap_fixed.h"
typedef ap_fixed<16,8> fx_t;

void gemm(fx_t A[][N],
          fx_t B[][N],
          fx_t C[][N]){
    for(int i=0;i<N;i++){
        for(int j=0;j<N;j++){
            fx_t acc=0;
            for(int k=0;k<N;k++){
                #pragma HLS PIPELINE
                acc+=A[i][k]*B[k][j];
            }
            C[i][j]=acc;
        }
    }
}
```

Float vs. Fixed: Full Comparison

Property	float	ap_fixed<16,8>
Adder latency	4 cy	1 cy
Adder LUT	~400	~16
Mul latency	3 cy	1–2 cy
Mul DSP	3	1
Range	$\pm 10^{38}$	± 128
Precision	24-bit M	8 frac. bits
II (accum.)	4	1
Code change	none	typedef

Use float when:

Wide dynamic range needed

Accuracy is critical

Area/speed not the bottleneck

Use ap_fixed when:

Targeting II = 1 on accumulation

Area budget is tight

Input range is bounded & known

Float vs. Fixed — Key Takeaways

1. Float is a pipeline, not a gate

A float adder has 4 stages. That latency propagates directly into II for any loop-carried sum.

2. ap_fixed breaks the dependency

1-cycle adder $\Rightarrow$ II = 1. Same pragma, same loop, one typedef change.

3. Choose W and I from data, not habit

Simulate in float, measure range, pick W.

	float	ap_fixed
II	4	1
LUT	1842	312
Latency	257 cy	66 cy

These numbers are for the GEMM inner loop, $N = 64$, on Zynq-7020 @ 100 MHz. Your lab results should be close.

L6 Key Takeaways

II determines throughput

Throughput = $1/(II \times T_{clk})$. Minimise II first.

Pragma order matters

PIPELINE $\rightarrow$ ARRAY_PARTITION $\rightarrow$ UNROLL.

Float costs area and II

ap_fixed recovers II=1, cuts LUT count.

Read the report

Let numbers drive every pragma decision.

Version	II	Speedup
Baseline	7	1 $\times$
+ PIPELINE (float)	4	$\approx 1.7\times$
+ ap_fixed + PIPELINE	1	7 $\times$
+ UNROLL $\times 4$	1	$\approx 28\times$

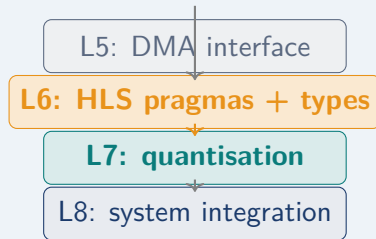
The final design is $\approx 28\times$ faster than baseline with lower LUT count than the float pipeline version.

What's Next — L7 Preview

L7 — Quantisation & Accuracy

- Measuring quantisation error
- Fixed-point simulation in C++
- Word-length optimisation methodology
- Case study: FIR filter in `ap_fixed`

Before L7: complete the lab — apply the pragmas, record synthesis reports, note where `ll` changes.



Questions?

L6 · HLS Optimisation & Float/Fixed Intro

José T. de Sousa · IST · 2025/26

$II = 1$

PIPELINE

ap_fixed

read report